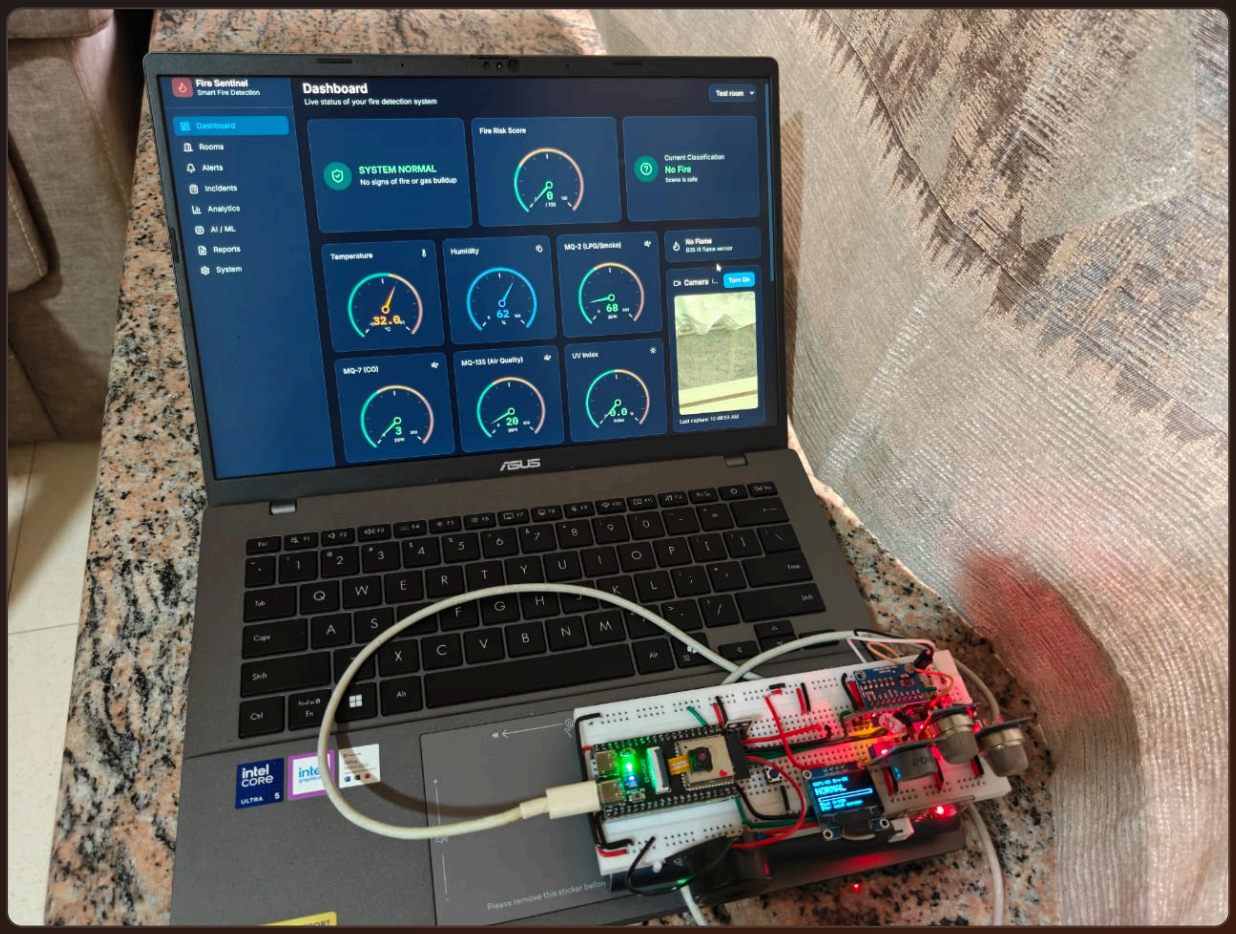
DESIGN EXPERIENCE · PROJECT REPORT

Fire Sentinel

An AI-assisted, multi-sensor fire detection and alerting system built on the ESP32-S3

github.com/kpr3306-lab/Fire_Sentinal

Edge sensor fusion · On-device CNN + YOLO · Flask backend · React dashboard · Telegram / Email alerts



SUBMITTED BY

Kshitij J046

Abish J047

Sohail J058

B.Tech Data Science

Semester V

November 2026

Contents

1 Introduction and problem statement
2 Objectives
3 System architecture
4 Hardware design
5 Firmware and detection logic
6 Machine learning
7 Backend and alerting
8 Web dashboard
9 Testing and results
10 Limitations and future work
11 Conclusion
12 Team and technology stack

Abstract

Conventional smoke detectors rely on a single sensor and a fixed threshold. They raise false alarms for burnt toast or steam, cannot distinguish a candle from a spreading fire, and cannot reach anyone who is away from home. **Fire Sentinel** (Prototype 2) addresses these gaps with a low-cost IoT device built around an ESP32-S3 camera board. It fuses readings from three gas sensors (MQ-2, MQ-7, MQ-135), an IR flame sensor, a UV sensor and a BME280 climate sensor into a transparent 0–100 risk score, which drives a four-stage alarm state machine that requires evidence to persist before escalating.

When the risk rises, the device captures a camera snapshot and classifies it on the board with two quantized neural networks: a MobileNetV2 CNN (INT8, 612 KB, **96.98 % test accuracy**, **91.92 % hazardous-fire recall**) and a YOLO11n flame detector. The alarm decision, buzzer and display work without a network connection. A Flask + SQLite backend stores the history, sends Telegram and Email alerts with the photo, and runs a simulated fire-department dispatch countdown, while a React dashboard shows live gauges, the camera, a detection timeline and analytics. The prototype has been assembled and demonstrated end to end, from live sensor readings to photo alerts on a phone. The full source code is available at https://github.com/kpr3306-lab/Fire_Sentinal.

1 Introduction and problem statement

Fire remains one of the most common household and workplace hazards, and early detection is the single largest factor in limiting damage. Yet the typical detector found in homes and hostels is an ionisation or photoelectric smoke alarm: one sensor, one threshold, and one loud buzzer. This design has three well-known weaknesses.

- **False alarms.** Cooking fumes, steam or a lit candle can cross the threshold. Frequent nuisance alarms lead people to disable detectors altogether.
- **No context.** The detector cannot tell a controlled flame (a stove, a diya, a candle) from a hazardous, spreading fire, and it cannot show anyone what it is reacting to.
- **No remote reach.** If nobody is home, the alarm sounds to an empty room. There is no notification, no evidence and no record of what happened.

Fire Sentinel was designed to keep the reliability of a local alarm while adding the intelligence and connectivity that modern, inexpensive hardware makes possible. It combines several independent physical signals, uses a camera and on-device AI to provide visual evidence, and connects to a backend that notifies the owner wherever they are.

2 Objectives

1. **Multi-sensor fusion.** Combine smoke/combustible gas, carbon monoxide, air quality (VOC), UV, IR flame and temperature into a single, explainable 0–100 risk score.
2. **Robust alarm logic.** Use a staged state machine (`NORMAL → PRECURSOR → CONFIRMING → FIRE_CONFIRMED`) that rejects short transients and only escalates on sustained, corroborated evidence.
3. **Visual evidence with edge AI.** Capture a snapshot on every escalation and classify it on the device as *no flame*, *controlled flame* or *hazardous fire*.
4. **Offline-first operation.** Keep detection, the buzzer and the display fully functional without WiFi.
5. **Remote awareness.** Send tiered Telegram and Email alerts with the photo, log every reading and event, and present them on a live web dashboard.
6. **Low cost and easy setup.** Use commodity modules, a single microcontroller board and self-registering devices that never need re-flashing to change rooms or IP addresses.

3 System architecture

Fire Sentinel is organised in three layers. The **device** senses, decides and alarms locally. The **backend** stores history, analyses snapshots and sends notifications. The **dashboard** presents everything to the user. Models are trained offline on a PC with the project's ML kit and shipped to the device on a microSD card.

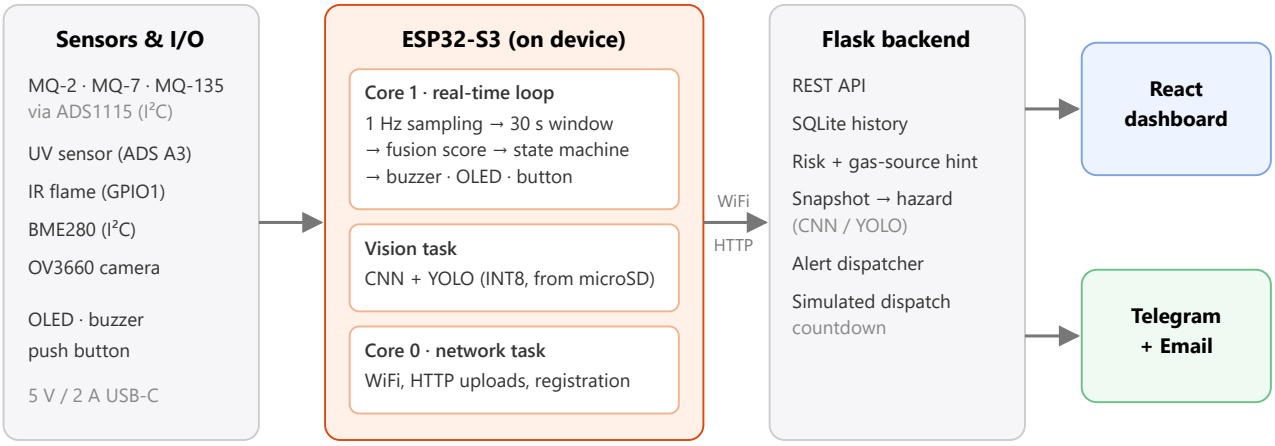


Figure 1. High-level architecture of Fire Sentinel.

Layer	Responsibility
ESP32-S3 device	Sample sensors, compute the risk score, decide the alarm state locally, sound the buzzer, show status on the OLED, classify snapshots, upload readings and photos.
Flask backend	Store readings and events, send alerts once per state change, analyse snapshots, run the simulated dispatch countdown and serve the REST API.
React dashboard	Live gauges and risk score, on-demand camera, snapshot gallery, detection timeline, rooms, alerts, analytics and reports.
ML kit (PC)	Dataset preparation, training, INT8 quantization and evaluation of the CNN, YOLO and sensor models.

A key design decision is the split of work across the ESP32-S3's two cores. All time-critical work (sampling, fusion, alarm state, buzzer and display) runs on core 1 and *never waits on the network*. Core 0 owns WiFi and HTTP through a small job queue. If the router or backend fails, detection carries on unaffected.

4 Hardware design

The prototype is built on a single Freenove ESP32-S3-WROOM CAM board (N16R8: 16 MB flash, 8 MB PSRAM, OV3660 camera, microSD slot). Because the camera occupies most of the on-chip ADC1 pins and ADC2 is unusable while WiFi is active, all analog sensors are read through an external 16-bit ADS1115 ADC on the I²C bus.

Component	Role in the system
Freenove ESP32-S3-WROOM CAM	Main controller, WiFi, camera, PSRAM for the neural networks, microSD for model files
MQ-2	Smoke and combustible gases (LPG, broad spectrum)
MQ-7	Carbon monoxide, a marker of incomplete combustion and smouldering
MQ-135	Air quality, VOCs and smoke
ADS1115 (0x48)	16-bit ADC reading the three MQ sensors (through 10 kΩ/10 kΩ dividers) and the UV sensor
UV sensor	Detects the ultraviolet signature emitted by flames
IR flame module (B35)	Digital flame detection on GPIO1, debounced in firmware
BME280 (0x76)	Temperature, humidity and pressure; temperature rise is a strong fire signal
SSD1306 OLED (0x3C)	Local status screens: state, risk bar, sensor values, network
Passive buzzer · push button	Distinct alarm pattern per state; short press cycles screens, long press silences and acknowledges

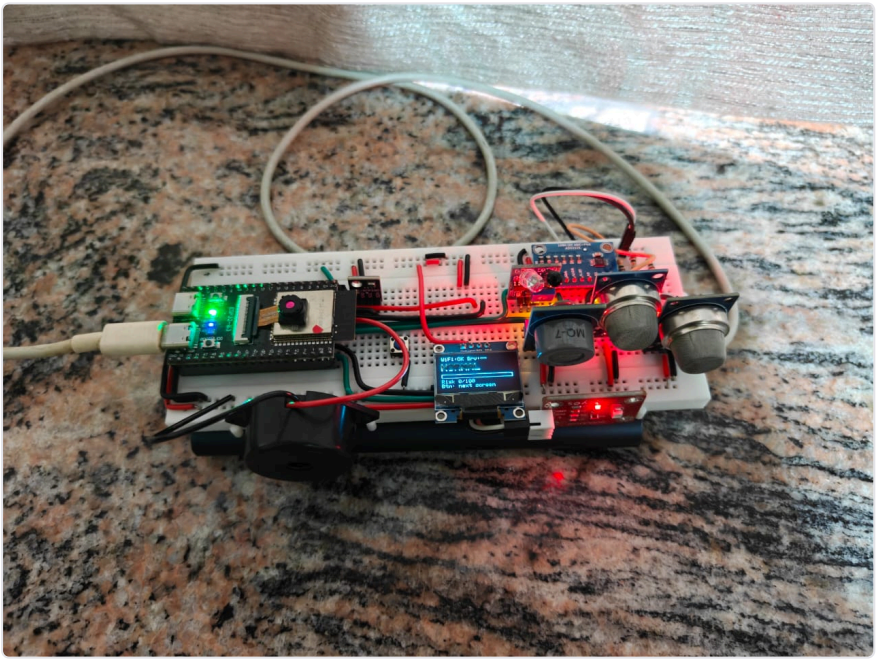


Figure 2. The assembled hardware prototype. The ESP32-S3 camera board is on the left; the MQ-7, MQ-2 and MQ-135 gas sensors are on the right; the OLED shows the device in the NORMAL state with a risk of 0/100.

Only the MQ heaters need 5 V; every other module runs on the 3.3 V rail, keeping the I²C bus at a safe logic level. The MQ outputs (up to 5 V) are halved by resistor dividers before reaching the ADS1115. The device is powered by a 5 V / 2 A USB-C adapter, since the three MQ heaters together draw roughly 450 mA.

5 Firmware and detection logic

5.1 Sampling, baseline and window features

The firmware samples every sensor at a fixed **1 Hz**. A fixed rate is deliberate: the rate-of-rise and variability features must mean the same thing during training and on the device. Each sample enters a 30-second ring buffer, from which the mean, standard deviation and least-squares slope of every channel are computed.

Gas sensors differ from unit to unit and drift over time, so Fire Sentinel judges readings *relative to the room's own baseline*, which is learned after a three-minute heater warm-up. The baseline is protected against slowly "learning" a fire as normal: learning freezes whenever any gas sensor is more than 25 % above baseline or the temperature moves by more than 1 °C, and the baseline is clamped around its first value. It is saved to flash every 30 minutes and restored at boot.

5.2 Fusion risk score

The risk score is rule-based and fully explainable: every point can be traced to a sensor.

Component	Max points	How the points are earned
Flame	35	Debounced IR flame detection (at least 4 of the last 10 polls active)
Temperature	25	Best of: deviation from baseline (+5 °C = full), rate of rise (+6 °C/min = full) or absolute level (≥ 57 °C = full)
Gas	30	Largest MQ deviation above baseline (+150 % = full)
Multi-gas agreement	10	Two or more MQ sensors above +40 %
UV	10	UV level above its baseline

The total is clamped to 100. Without a visible flame the score can still reach 75, so a smouldering fire is able to escalate on gas and heat alone. A temperature of 57 °C or more forces at least the CONFIRMING level, mimicking a fixed-temperature heat detector.

5.3 Alarm state machine

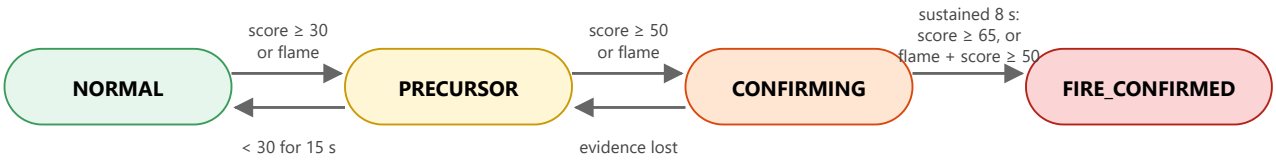


Figure 3. The four-stage alarm state machine. A snapshot is captured on entry to each elevated state.

Escalation to FIRE_CONFIRMED requires fire-level evidence to persist for **8 seconds**. A flame signal alone (for example sunlight or a lamp) can hold the device at CONFIRMING, which already alerts the

owner, but cannot escalate to a full fire on its own. A lighter flick never satisfies the 8-second window, and the 15-second cooldown lets brief disturbances such as burnt toast settle back to NORMAL.

State	Buzzer pattern	Notifications
NORMAL	Silent	Heartbeat upload every 30 s
PRECURSOR	One short beep every 6 s	Telegram
CONFIRMING	Double beep every 1.5 s	Telegram + Email
FIRE_CONFIRMED	Continuous tone	Telegram + Email + simulated dispatch countdown

The device registers itself with the backend using an ID derived from its MAC address, so rooms and IP addresses are assigned from the dashboard without re-flashing. A runtime I²C health check flags a disconnected sensor, excludes it from the score and recovers automatically when it is reconnected.

6 Machine learning

Fire Sentinel uses three models with distinct jobs. The two vision models run on the ESP32-S3 itself, loaded from the microSD card into PSRAM.

Model	Input	Output	Runs on
CNN · MobileNetV2 α = 0.35, 96×96, INT8	Camera frame	NO_FLAME / CONTROLLED_FLAME / HAZARDOUS_FIRE	ESP32-S3
YOLO11n, single class "fire", INT8 @ 160 px	Camera frame	Flame bounding boxes, size and growth	ESP32-S3 and backend
RandomForest + IsolationForest	30 s window of 16 sensor features	Class + "not normal" flag	ESP32-S3 (compiled into flash)

6.1 CNN snapshot classifier

Data. 21,028 images were collected and cleaned: 1,322 duplicates and 79 cross-class label conflicts were removed, leaving 19,632 images. These were split into 13,743 training, 2,945 validation and 2,944 test images. The split is *grouped*, so near-identical images never appear in both training and test, which keeps the reported accuracy honest.

Training. Transfer learning from an ImageNet-pretrained MobileNetV2 (width multiplier 0.35, 96×96 input) with class weights to counter the smaller hazardous class (weight 2.97 vs. about 0.8 for the others). Early stopping ended training after 23 epochs, in about seven minutes.

Quantization. Full-integer post-training quantization with 300 representative images produced an INT8 model small enough for the microcontroller.

Model	Size	Accuracy	Macro F1	Hazardous recall	False hazard alarm
Keras FP32	4,543 KB	97.76 %	0.970	91.52 %	0.78 %
TFLite FP32	1,573 KB	97.76 %	0.970	91.52 %	0.78 %
TFLite INT8 (deployed)	**612 KB**	**96.98 %**	**0.961**	**91.92 %**	**1.39 %**

2.57×
smaller than FP32 TFLite

−0.78 pt
accuracy cost of INT8

99.3 %
controlled-flame F1

2,944
held-out test images

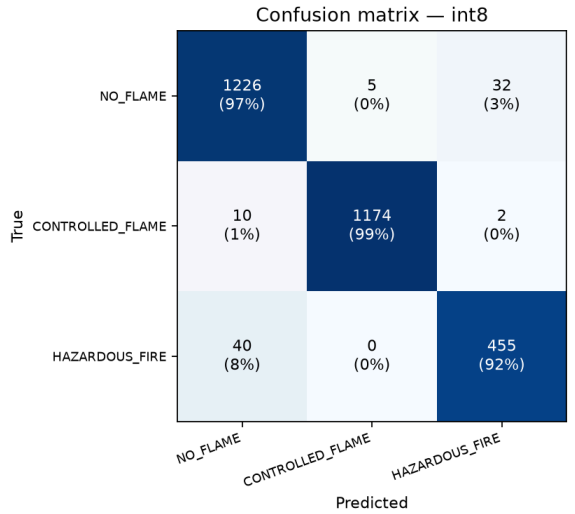


Figure 4. INT8 CNN confusion matrix on the test set.

The dominant error is a hazardous fire predicted as no flame (40 of 495, 8.1 %), typically scenes of heavy smoke with little visible flame. For this reason the camera is used only to **raise** confidence, never to lower the sensor-driven risk.

Using a threshold on P(hazardous) instead of the arg-max allows a recall/false-alarm trade-off: a threshold of 0.20 gives 96.4 % recall at a 3.1 % false-alarm rate, while 0.50 gives 92.3 % at 1.5 %. For a safety system, higher recall is the preferred side of that trade-off.

6.2 YOLO11n flame detector

A single-class detector was built from public Roboflow fire datasets and trained on a Google Colab GPU for 50 epochs at 640 px (batch 16, starting from `yolo11n.pt`). Final validation scores were **precision 0.79, recall 0.61, mAP@50 0.69 and mAP@50-95 0.34**. It was exported to INT8 at 160×160 (2.9 MB) for the ESP32-S3.

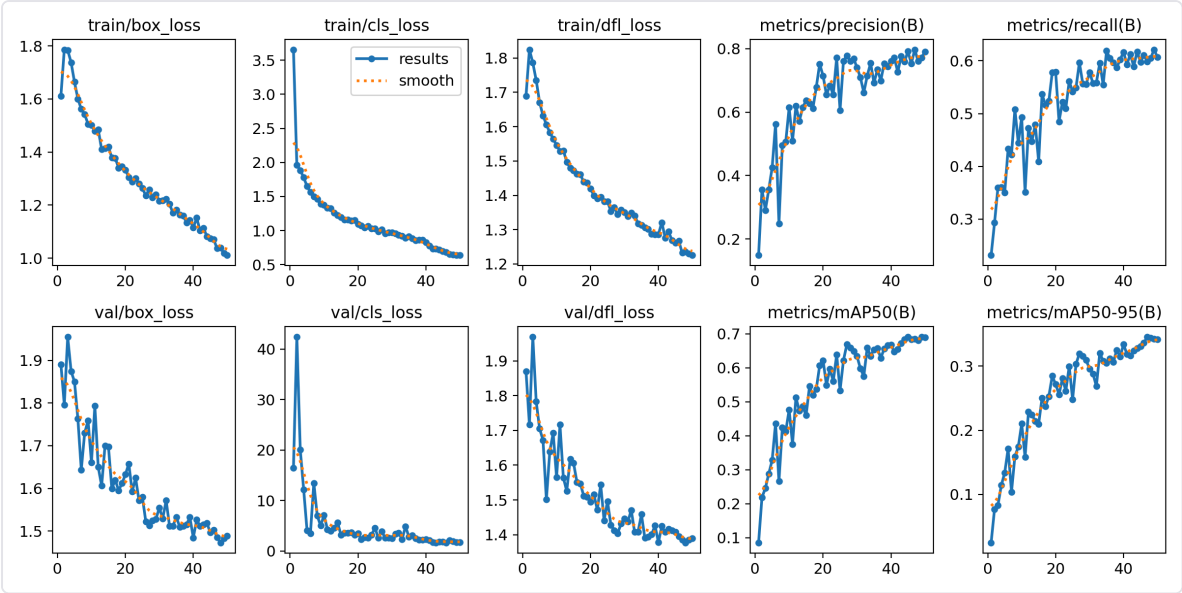


Figure 5. YOLO11n training and validation curves over 50 epochs.

YOLO is a *secondary* signal. A snapshot is marked hazardous when the flame area covers at least 15 % of the frame or grows by 1.5× within three minutes. Because box recall is limited, "no boxes found" is never treated as proof that there is no fire.

6.3 Sensor model

A RandomForest classifier (40 trees, depth ≤ 8) and an IsolationForest anomaly detector operate on 16 baseline-relative window features (temperature deviation, spread and slope; relative mean, spread and slope of each MQ sensor; UV statistics; flame duty). The IsolationForest is trained on normal data only, so it can flag "this is not normal" even for fire types never recorded. Evaluation uses leave-one-session-out validation, because neighbouring 30-second windows overlap by 29 samples and a random split would leak. The trained trees are exported to a C header and compiled into firmware flash. A parity test compiles the actual firmware inference code and confirms it matches scikit-learn on 678 windows (100 % agreement). The pipeline is complete, but it has so far been trained on synthetic data only, so the device currently ships with the rule-based fusion score.

7 Backend and alerting

The backend is a Flask REST API with an SQLite database that migrates its own schema at start-up. Each reading posted by the device is stored together with raw sensor counts, window statistics and model outputs. A rule-based **gas-source hint** reports which MQ sensor is most elevated (for example, "possible carbon monoxide build-up").

7.1 Tiered alerts

Alerts are sent **once per state change**, not on every upload, so the one-second upload rate during an incident never floods the user.

State	Telegram	Email	Severity
PRECURSOR	✓	–	Info
CONFIRMING	✓	✓	Warning
FIRE_CONFIRMED	✓	✓	Critical

7.2 Snapshot hazard pipeline

Each uploaded snapshot passes through the CNN classification (taken from the board when the backend model is absent), YOLO flame detection and a hazard assessment that yields *HAZARDOUS*, *CONTROLLED*, *NONE* or *UNKNOWN*. Escalation is one-way: any single hazardous signal wins.

7.3 Simulated fire-department dispatch

On FIRE_CONFIRMED the owner receives a Telegram message and a **30-second countdown** starts. It can be cancelled by long-pressing the device button or from the API. If nobody cancels, a message clearly marked *[SIMULATION — no real call was placed]*, containing the room, risk, camera verdict and photo, is sent to a separate Telegram chat. A background timer ensures the countdown completes even if the device goes offline mid-fire.

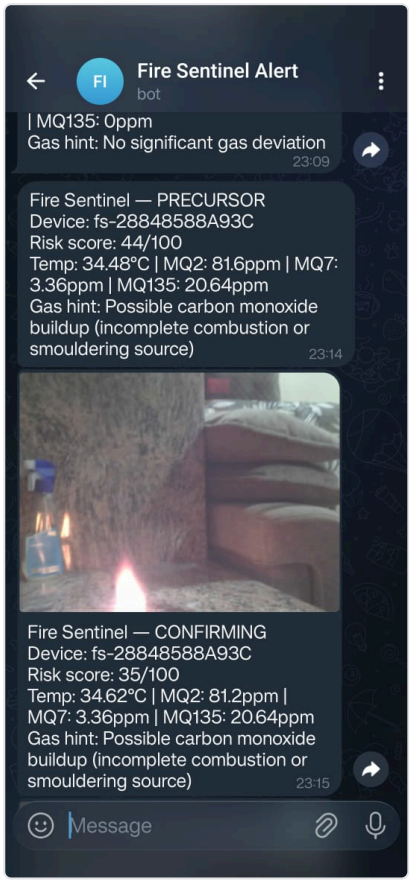


Figure 6. Live Telegram alerts during a test: a PRECURSOR alert with a CO gas hint, followed by a CONFIRMING alert carrying the on-board camera snapshot of the flame.

> **Safety by design.** Fire Sentinel never places a real emergency call. An unverified automated call is a liability and a source of false dispatches. A production deployment would escalate to a verified personal contact or a certified monitoring partner.

8 Web dashboard

The dashboard is built with React, Vite and Tailwind CSS. The home page shows the system status, a live fire-risk gauge, the current camera classification and custom SVG gauges for temperature,

humidity, each gas sensor and UV. A camera panel captures images on demand through the backend proxy.



Figure 7. The dashboard home page showing a NORMAL state, risk score 4/100 and live sensor gauges.

Page	What it provides
Dashboard	Status, risk gauge, classification, sensor gauges, camera panel, snapshot gallery, detection timeline
Rooms	Room profiles, appliances and emergency contacts
Alerts · Incidents	Alert history per channel with delivery status; incidents grouped from events
Analytics · Reports	Sensor trend charts and exportable (CSV) reports
AI / ML	Model information from the CNN manifest
System	Registered devices, room assignment and manual IP override

9 Testing and results

Testing was carried out at three levels: host-side unit tests of the firmware logic, automated backend tests, and end-to-end trials on the assembled hardware.

What was tested	Method	Result
Fusion score, state machine, baseline freeze and clamp, window slope, flame debounce	Host-side C++ tests (g++)	9 scenarios pass: smoulder without flame, flame only, flame + gas, transients, 60 °C floor, slow creep
Buzzer silence and re-arm, button debounce and long press	Host-side C++ tests	13 checks pass
Firmware sensor-model inference vs. scikit-learn	Parity test compiling the real firmware code	678 windows, 100 % agreement, max probability difference 1e-4
Backend: hazard pipeline, dispatch, sensor ingest	`unittest` suite	29 tests pass (9 + 11 + 9)
Firmware build	Arduino ESP32 core 3.3.12	Compiles; uses 46 % of the Huge APP partition
CNN classifier	Held-out test set, FP32 vs. INT8	96.98 % accuracy, 91.92 % hazardous recall (INT8)
End-to-end on hardware	Live trials with a controlled flame	Live readings on the dashboard; PRECURSOR and CONFIRMING alerts delivered to Telegram with the board's snapshot (Figure 6)

9.1 Scenario walk-throughs

Scenario	System behaviour
Quiet night	1 Hz sampling, baseline tracks slowly, NORMAL state, buzzer silent, OLED asleep, 30 s heartbeat.
Burnt toast	Gas rises → PRECURSOR (short beep, snapshot, Telegram). Air clears → after 15 s below threshold, back to NORMAL.
Smouldering fire	All three MQ sensors rise with temperature → CONFIRMING (Telegram + Email with photo) → sustained ≥ 65 → FIRE_CONFIRMED and dispatch countdown.
Candle or lighter	A brief flick never fills the 8 s window. A burning candle reaches CONFIRMING but not FIRE_CONFIRMED without gas or heat; the CNN labels it a controlled flame.
Owner on site	Long-press the button → buzzer silenced, acknowledgement sent, dispatch cancelled.
WiFi or backend down	Detection, buzzer and OLED continue; the device reconnects and re-registers automatically.

10 Limitations and future work

Current limitations

- Fusion weights and thresholds (30 / 50 / 65) are hand-tuned. The sensor ML model has been trained on synthetic data only, so the device uses the rule-based score.

- MQ sensors are not gas-specific, drift with temperature, humidity and age, and need a 24–48 h burn-in. The MQ-7 is not heater-cycled, which reduces its CO sensitivity.
- YOLO's on-board accuracy at 160 px is likely below its 640 px validation figure and has not yet been re-measured at the deployed resolution.
- Failed uploads are dropped rather than buffered, and the live camera proxy requires the backend to be on the same LAN as the device.
- The fire-department dispatch is a simulation.

Future work

- Record real, labelled sensor sessions (normal activity, controlled flames, small supervised outdoor burns) and train the sensor model, then use it to re-tune the fusion thresholds.
- Re-evaluate YOLO at 160 px and tune the P(hazardous) alarm threshold for higher recall.
- Buffer readings on the microSD card while offline and upload them on reconnect.
- Add phone-call escalation (Twilio) to a verified personal contact and a dashboard panel for the dispatch countdown.
- Design an enclosure with proper airflow and support multiple devices per home.

11 Conclusion

Fire Sentinel shows that an inexpensive microcontroller can deliver a substantially smarter fire detector than a conventional single-sensor alarm. Fusing six physical signals into an explainable risk score, and requiring evidence to persist through a staged state machine, reduces nuisance alarms while still catching smouldering fires that have no visible flame. Quantizing a MobileNetV2 classifier to 612 KB with less than one percentage point of accuracy loss made it possible to classify snapshots directly on the ESP32-S3, giving every alert visual evidence without depending on the cloud. The backend and dashboard turn the device into a connected system that reaches the owner anywhere, while the offline-first design ensures that the core safety function never depends on the network. With real-world sensor data and further field testing, the same architecture can mature from a student prototype into a dependable home-safety product.

12 Team and technology stack

Name	Roll number
Kshitij	J046
Abish	J047
Sohail	J058

B.Tech Data Science, Semester V

Source code: https://github.com/kpr3306-lab/Fire_Sentinal

Area	Technologies
Hardware	ESP32-S3 (Freenove WROOM CAM), MQ-2/7/135, ADS1115, BME280, UV and IR flame sensors, SSD1306 OLED
Firmware	Arduino (ESP32 core 3.x), FreeRTOS tasks, TensorFlow Lite Micro, ArduinoJson 7
Machine learning	Python, TensorFlow/Keras (MobileNetV2), Ultralytics YOLO11, scikit-learn
Backend	Python, Flask, SQLite, Telegram Bot API, Gmail SMTP
Dashboard	React, Vite, Tailwind CSS

Disclaimer. Fire Sentinel is an academic prototype and is not a certified fire alarm. Hardware trials used only a small, controlled flame.